

ЛАБОРАТОРНАЯ РАБОТА №1

«ИССЛЕДОВАНИЕ АЛГОРИТМОВ ПРОГРАММИРОВАНИЯ ДЛЯ ВЫПОЛНЕНИЯ ОПЕРАЦИЙ НАД ЛИНЕЙНЫМИ СПИСКАМИ НА ЯЗЫКЕ C/C++»

1. Цель работы

Изучение списковых структур данных и приобретение навыков разработки и отладки программ, использующих динамическую память. Исследование особенностей организации списков средствами языка C/C++.

2. Постановка задачи и вариант задания

Структуру данных представить в виде линейного списка, элементами которого являются строки таблицы. Написать функции:

- организации списка;
- просмотра списка;
- добавления элемента в список;
- исключения элемента из списка;
- освобождения динамической памяти (обязательно вызывать при выходе из программы);
- одну из функций в соответствии с вариантом, приведенным ниже.

Значения и количество записей в таблице пользователь выбирает самостоятельно (т.е. количество строк таблицы не задается). Работу программы необходимо оформить в виде меню.

Вариант 4: Даны сведения об ученых. Структура записи: уникальный ID, ФИО ученого, научная область, ученая степень, количество опубликованных статей, индекс Хирша, количество цитирований.

Функцию, которая удаляет из непустого списка два элемента, после первого, т.е. удаляет второй и третий элемент.

3. Краткие теоретические сведения

Для выполнения лабораторной работы необходимо изучить особенности объявления, описания и вызова линейных списков структур данных в языках C/C++.

ХОД РАБОТЫ

4. Структурная схема алгоритма

Структурная схема алгоритма, а также всех функций представлены на рисунках 1.1 – 1.9:

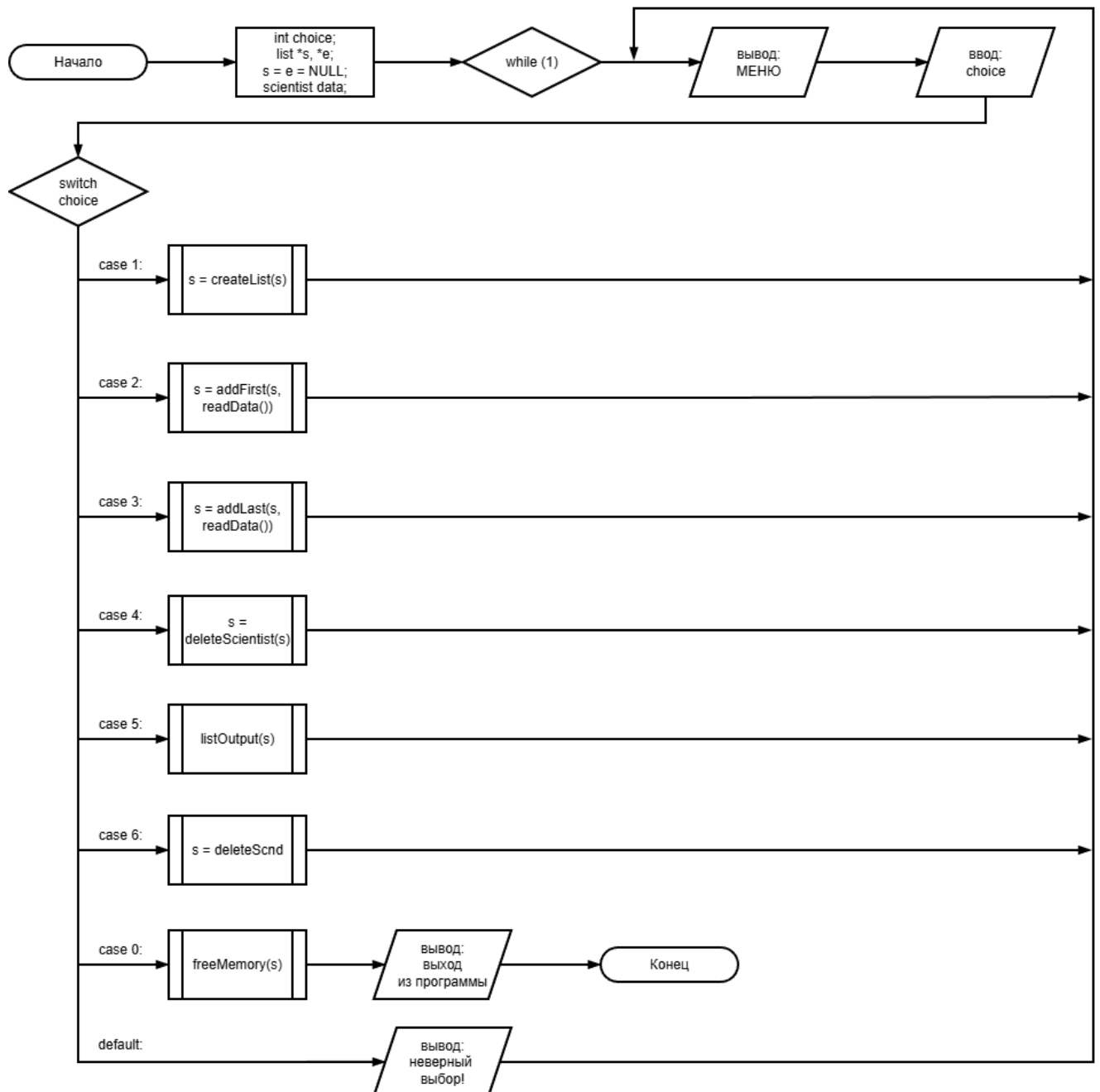


Рисунок 1.1 – Структурная схема функции `main()`

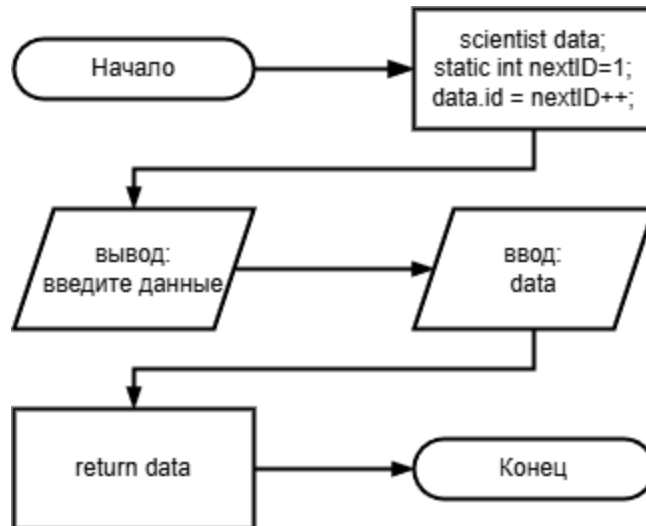


Рисунок 1.2 – Структурная схема функции *readData()*

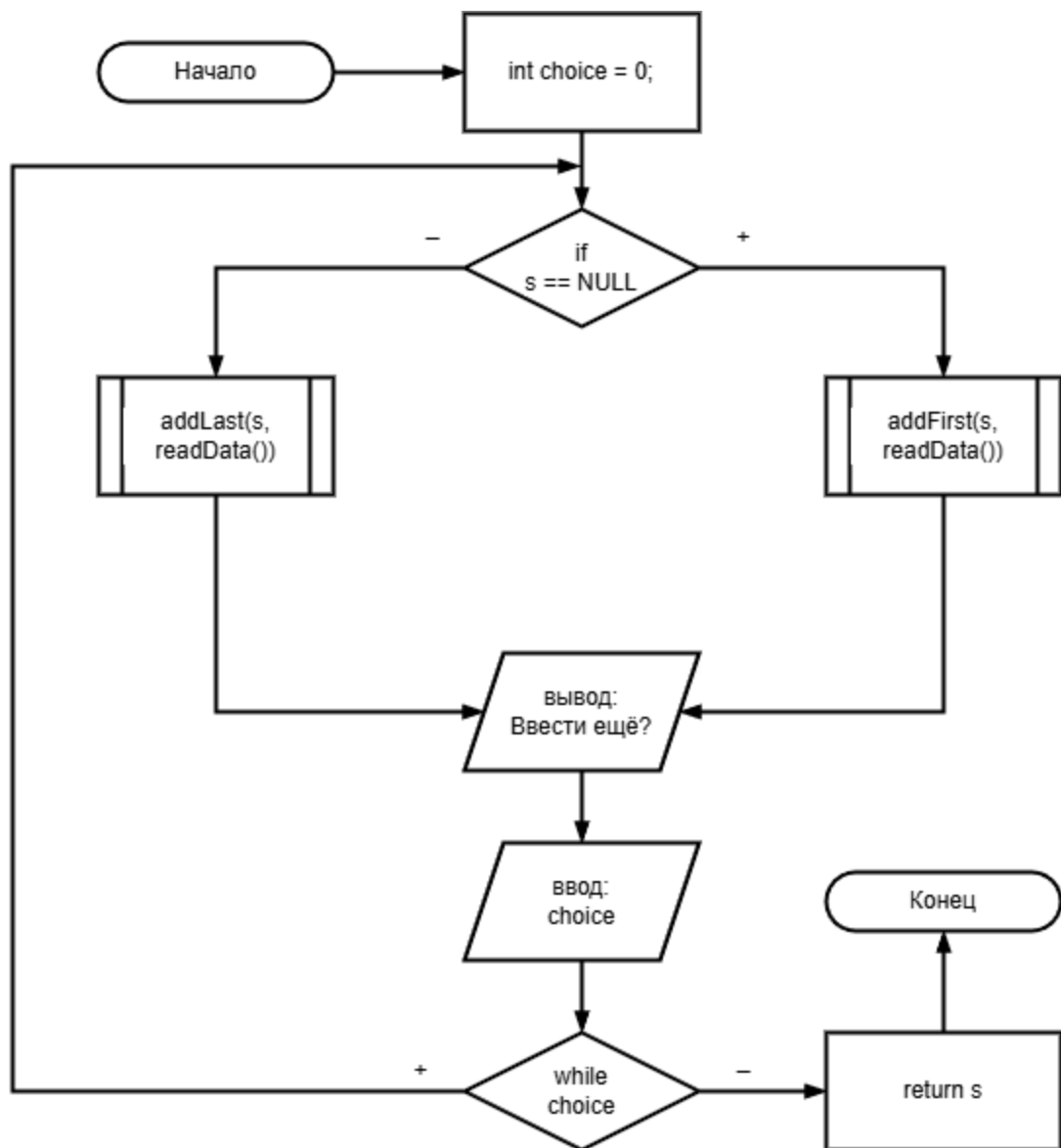


Рисунок 1.3 – Структурная схема функции *createList(*s)*

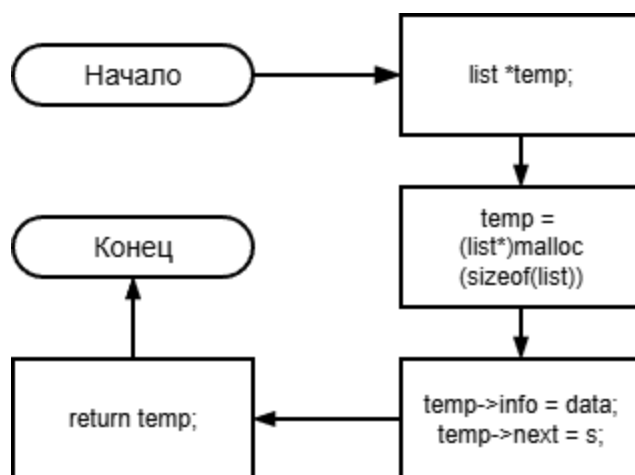


Рисунок 1.4 – Структурная схема функции `addFirst(*s, data)`

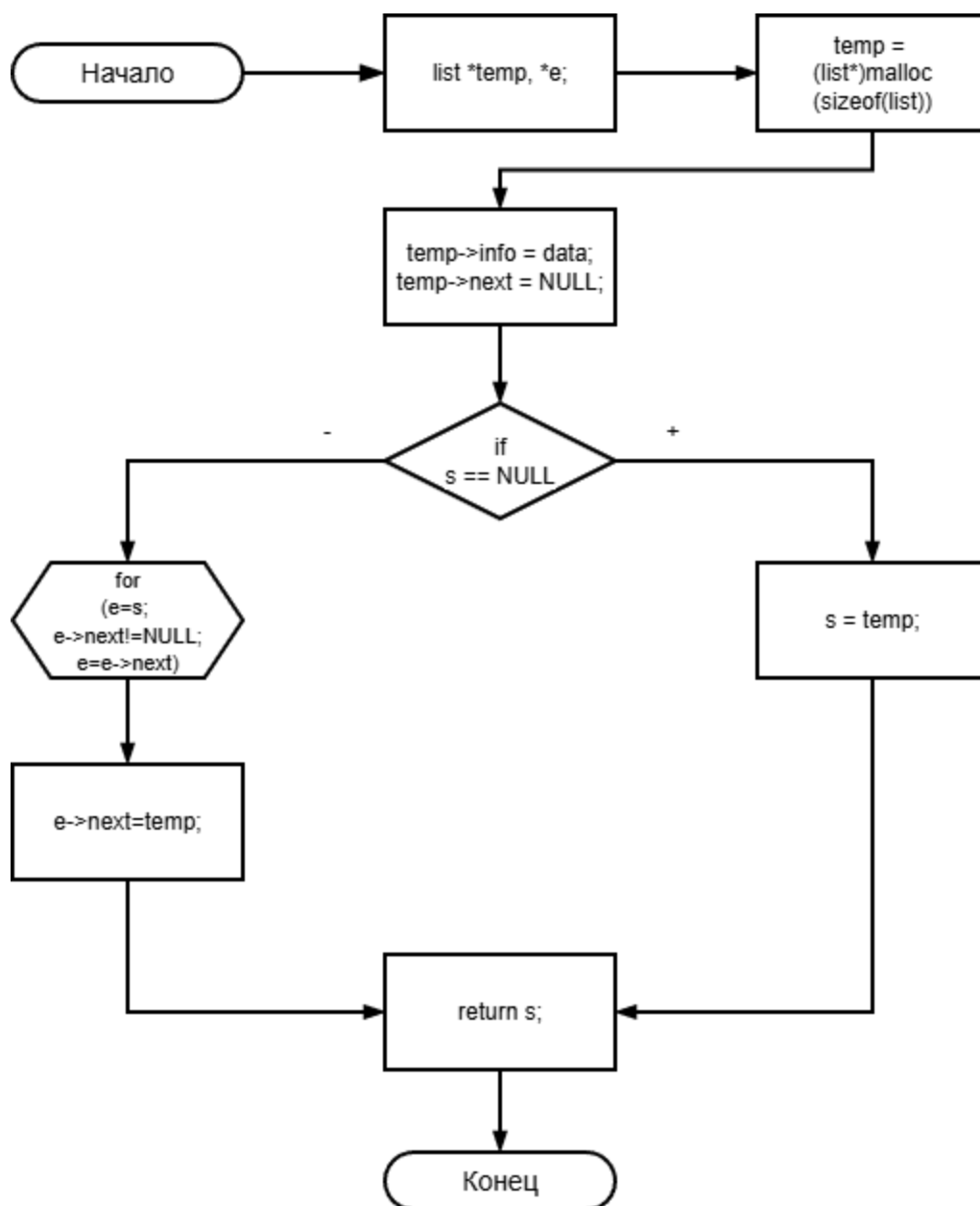


Рисунок 1.5 – Структурная схема функции `addLast(*s, data)`

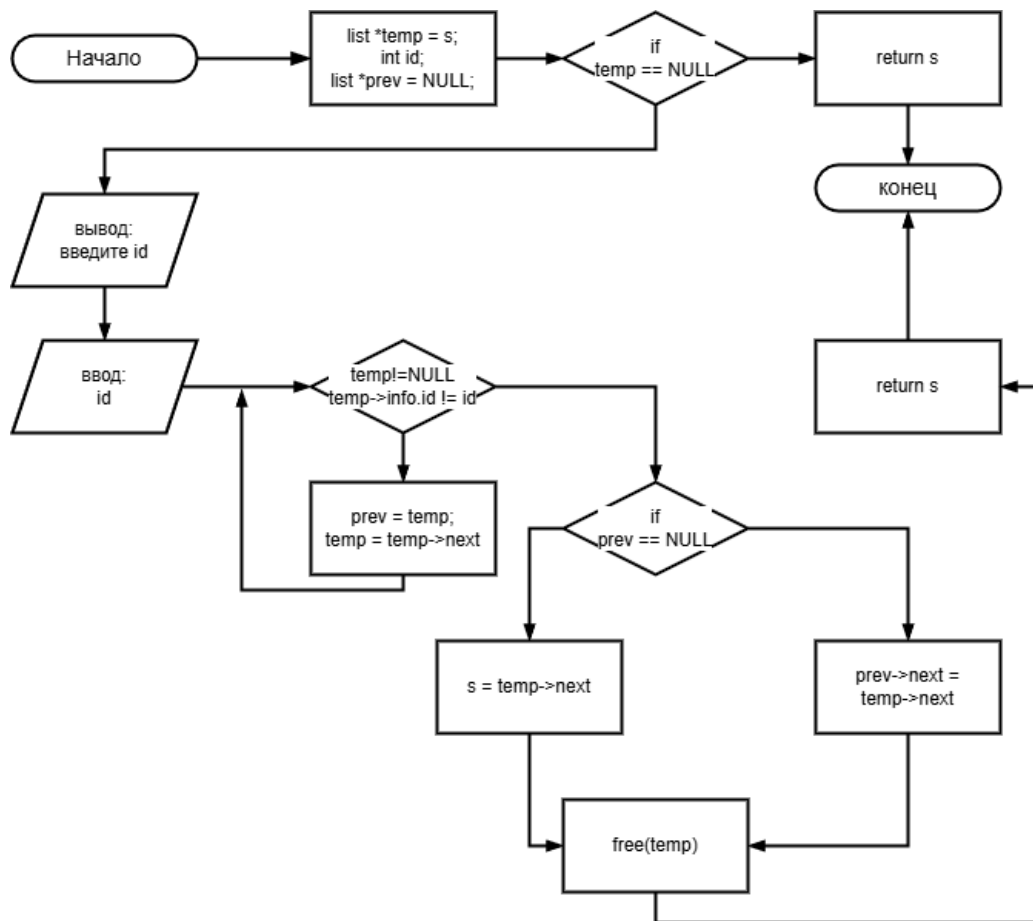


Рисунок 1.6 – Структурная схема функции deleteScientist(*s)

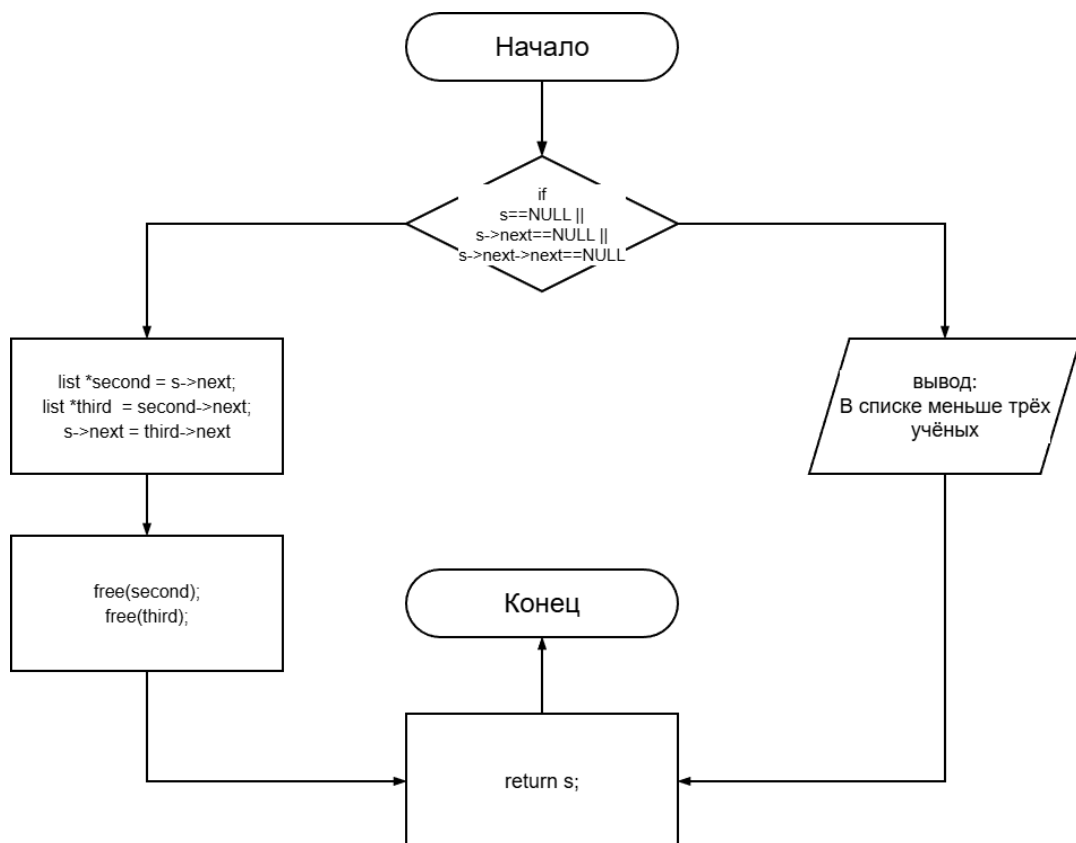


Рисунок 1.7 – Структурная схема функции deleteScndThrdScientist(*s)

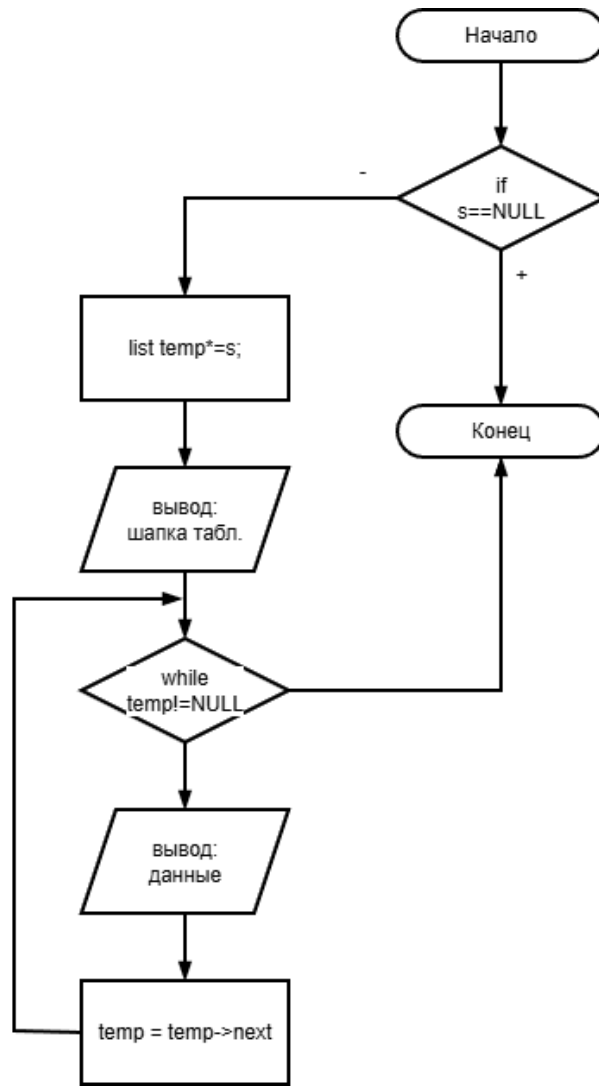


Рисунок 1.8 – Структурная схема функции `listOutput(*s)`

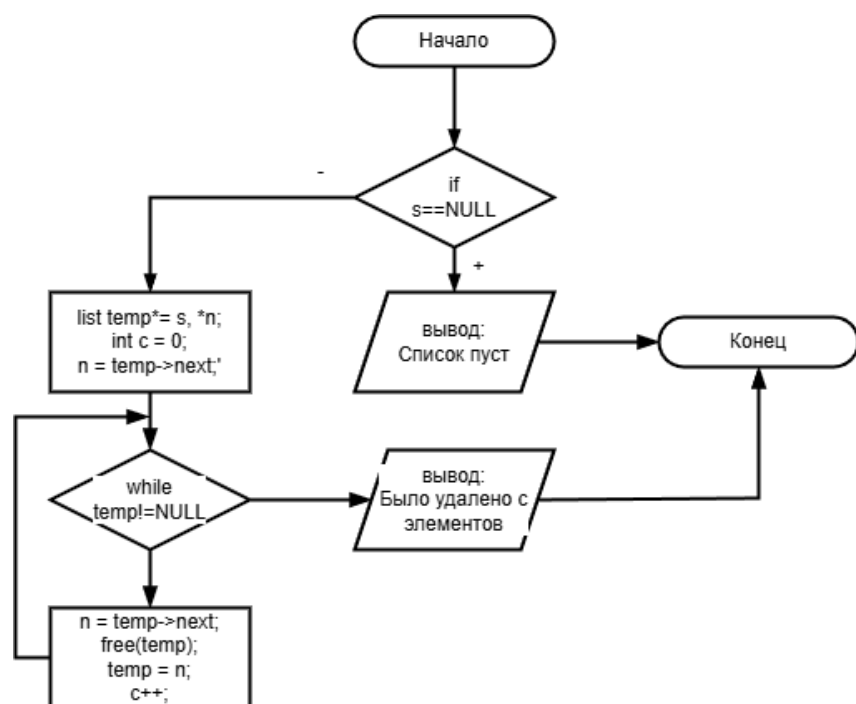


Рисунок 1.9 – Структурная схема функции `freeMemory(*s)`

5. Текст кода на языке С. Тестирование и отладка программы

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>

struct scientist {
    int id;
    char name[76];
    char area[26];
    char degree[26];
    unsigned int articles;
    unsigned int quotes;
    unsigned int hirshIndex;
};

struct list {
    struct scientist info;
    struct list* next;
};

struct scientist readData();
struct list *createList(struct list *s);
struct list *addFirst(struct list *s, struct scientist d);
struct list *addLast(struct list *s, struct scientist d);
struct list *deleteScientist(struct list *s);
struct list *deleteScndThrdScientist(struct list *s);
void listOutput(struct list *s);
void freeMemory(struct list *s);

int main() {
    int choice;
    struct list *s;
    s = NULL;
    struct scientist data;
    while (1) {
        printf("\n===== МЕНЮ =====\n");
        printf("1. Создать список учёных\n");
        printf("2. Ввести информацию об учёном в начало\n");
        printf("3. Ввести информацию об учёном в конец\n");
        printf("4. Удалить учёного из списка\n");
        printf("5. Вывести список всех учёных\n");
        printf("6. Удалить 2 и 3 запись\n");
        printf("0. Выход из программы\n");
        printf("-> ");
        scanf("%d", &choice);

        switch (choice) {
            case 1:
                s = createList(s);
                break;

            case 2:
                s = addFirst(s, readData());
```

```

        break;

    case 3:
        s = addLast(s, readData());
        break;

    case 4:
        s = deleteScientist(s);
        break;

    case 5:
        listOutput(s);
        break;

    case 6:
        s = deleteScndThrdScientist(s);
        break;

    case 0:
        freeMemory(s);
        printf("Выход из программы...");
        return 0;

    default:
        printf("Команда не распознана!\n");
    }
}

struct scientist readData() {
    struct scientist data;
    static int nextID = 1;
    data.id = nextID++;
    printf("Введите ФИО: ");
    gets(data.name);
    printf("Введите научную область: ");
    gets(data.area);
    printf("Введите учёную степень: ");
    gets(data.degree);
    printf("Введите количество научных статей: "); scanf("%d",
&data.articles);
    printf("Введите количество цитирований: "); scanf("%d",
&data.quotes);
    printf("Введите индекс Хирша: "); scanf("%d",
&data.hirshIndex);
    printf("\n");
    return data;
}

struct list *createList(struct list *s) {
    int choice = 0;
    do {
        if (s == NULL) {

```



```

        s = addFirst(s, readData());
    }
    else {
        addLast(s, readData());
    }
    printf("Ввести ещё? \n1. да \n0. нет \n-> ");
    scanf("%d", &choice);
} while (choice);
return s;
}

struct list *addFirst(struct list *s, struct scientist data) {
    struct list *temp;
    temp = (struct list*)malloc(sizeof(struct list));
    temp->info = data;
    temp->next = s;
    return temp;
}

struct list *addLast(struct list *s, struct scientist data) {
    struct list *temp, *e;
    temp = (struct list*)malloc(sizeof(struct list));
    temp->info = data;
    temp->next = NULL;
    if (s == NULL) s = temp;
    else {
        for (e = s; e->next != NULL; e = e->next);
        e->next = temp;
    }
    return s;
}

struct list *deleteScientist(struct list *s) {
    struct list *temp = s, *prev = NULL;
    int id;
    if (temp == NULL) return s;
    printf("Введите ID учёного, которого хотите удалить: ");
    scanf("%d", &id);
    while (temp != NULL && temp->info.id != id) {
        prev = temp;
        temp = temp->next;
    }
    if (prev == NULL) {
        s = temp->next;
    } else {
        prev->next = temp->next;
    }
    free(temp);
    return s;
}

struct list *deleteScndThrdScientist(struct list *s) {
    if (s == NULL || s->next == NULL || s->next->next == NULL) {

```

```

        printf("В списке меньше трёх учёных! Удаление
невозможно.\n");
        return s;
    }
    struct list *second = s->next;
    struct list *third = second->next;
    s->next = third->next;
    free(second);
    free(third);
    return s;
}

void listOutput(struct list *s) {
    if (s == NULL) {
        printf("Список пуст!\n");
        return;
    }
    struct list *temp;
    temp = s;
    printf("+-----+-----+-----+-----+-----+-----+
-----+-----+-----+-----+-----+-----+
-----+-----+-----+-----+-----+-----+
-----+\n");
    printf("| ID | Фамилия Имя
Отчество | Учёная Степень |
Область Науки | Кол-во статей | Кол-во цитирований | Индекс Хирша
|\n");
    printf("+-----+-----+-----+-----+-----+-----+
-----+-----+-----+-----+-----+-----+
-----+-----+-----+-----+-----+-----+
-----+\n");
    while (temp != NULL) {
        printf("| %-4d | %-75s | %-25s | %-25s | %-13d | %-18d |
%-12d |\n",
            temp->info.id,
            temp->info.name,
            temp->info.degree,
            temp->info.area,
            temp->info.articles,
            temp->info.quotes,
            temp->info.hirshIndex
        );
        temp = temp->next;
        printf("+-----+-----+-----+-----+-----+-----+
-----+-----+-----+-----+-----+-----+
-----+-----+-----+-----+-----+-----+
-----+\n");
    }
}

void freeMemory(struct list *s) {
    if (s == NULL) {
        printf("Список пуст...\n");
    }
}

```

```

        return;
    }
    struct list *temp=s, *n;
    int c = 0;
    n = temp->next;
    while (temp != NULL) {
        n = temp->next;
        free(temp);
        temp = n;
        c++;
    }
    printf("Было удалено %d записей...\nСписок пуст...\n", c);
}

```

На Рисунках 1.10, 1.11, 1.12 и 1.13 отображены результаты работы кода согласно всем условиям по заданию для языка С.

```

===== МЕНЮ =====
1. Создать список учёных
2. Ввести информацию об учёном в начало
3. Ввести информацию об учёном в конец
4. Удалить учёного из списка
5. Вывести список всех учёных
6. Удалить 2 и 3 запись
0. Выход из программы
-> 1
Введите ФИО: Andrew Vladislav Goldberg
Введите научную область: Math/Comp Sci
Введите учёную степень: Master of Science
Введите количество научных статей: 90
Введите количество цитирований: 8082
Введите индекс Хирша: 41

Ввести ещё?
1. да
0. нет
-> 1
Введите ФИО: Viktor Ginzburg
Введите научную область: Mathematics
Введите учёную степень: Doctor of Philosophy
Введите количество научных статей: 77
Введите количество цитирований: 4347
Введите индекс Хирша: 31

Ввести ещё?
1. да
0. нет
-> 1
Введите ФИО: Bjarne Stroustrup
Введите научную область: Computer Science
Введите учёную степень: Doctor of Science
Введите количество научных статей: 100
Введите количество цитирований: 23266
Введите индекс Хирша: 47

Ввести ещё?
1. да
0. нет
-> 0

===== МЕНЮ =====
1. Создать список учёных
2. Ввести информацию об учёном в начало
3. Ввести информацию об учёном в конец
4. Удалить учёного из списка
5. Вывести список всех учёных
6. Удалить 2 и 3 запись
0. Выход из программы

```

Рисунок 1.10 – Результат работы программы на языке С

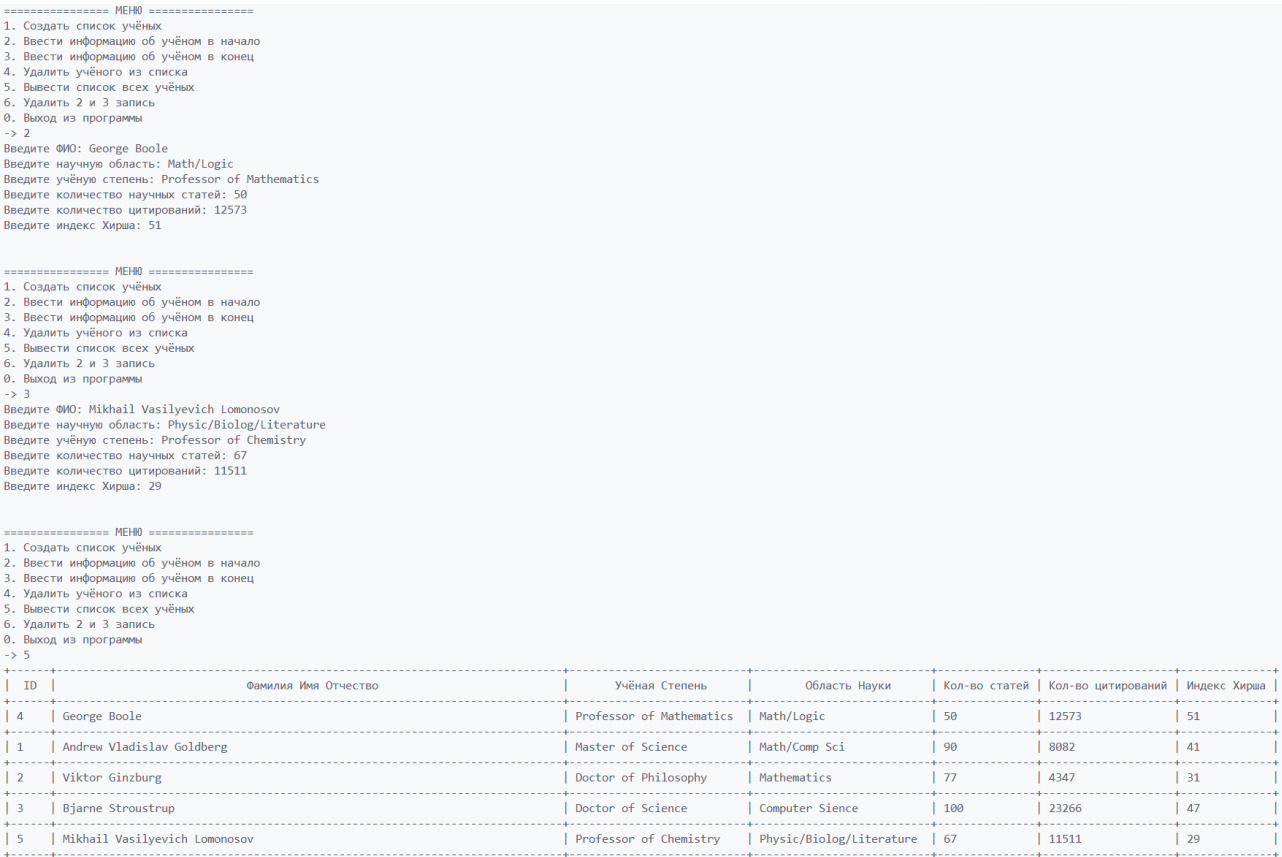


Рисунок 1.11 – Результат работы программы на языке C



Рисунок 1.12 – Результат работы программы на языке C

```
===== MENU =====
1. Создать список учёных
2. Ввести информацию об учёном в начало
3. Ввести информацию об учёном в конец
4. Удалить учёного из списка
5. Вывести список всех учёных
6. Удалить 2 и 3 запись
0. Выход из программы
-> 5

+-----+-----+-----+-----+-----+-----+
| ID |          Фамилия Имя Отчество          |   Учёная Степень   |   Область Науки   | Кол-во статей | Кол-во цитирований | Индекс Хирша |
+-----+-----+-----+-----+-----+-----+
| 4 | George Boole | Professor of Mathematics | Math/Logic | 50 | 12573 | 51 |
+-----+-----+-----+-----+-----+-----+
| 1 | Andrew Vladislav Goldberg | Master of Science | Math/Comp Sci | 90 | 8082 | 41 |
+-----+-----+-----+-----+-----+-----+
| 3 | Bjarne Stroustrup | Doctor of Science | Computer Science | 100 | 23266 | 47 |
+-----+-----+-----+-----+-----+-----+
| 5 | Mikhail Vasilyevich Lomonosov | Professor of Chemistry | Physic/Biolog/Literature | 67 | 11511 | 29 |
+-----+-----+-----+-----+-----+-----+

===== MENU =====
1. Создать список учёных
2. Ввести информацию об учёном в начало
3. Ввести информацию об учёном в конец
4. Удалить учёного из списка
5. Вывести список всех учёных
6. Удалить 2 и 3 запись
0. Выход из программы
-> 6

===== MENU =====
1. Создать список учёных
2. Ввести информацию об учёном в начало
3. Ввести информацию об учёном в конец
4. Удалить учёного из списка
5. Вывести список всех учёных
6. Удалить 2 и 3 запись
0. Выход из программы
-> 5

+-----+-----+-----+-----+-----+-----+
| ID |          Фамилия Имя Отчество          |   Учёная Степень   |   Область Науки   | Кол-во статей | Кол-во цитирований | Индекс Хирша |
+-----+-----+-----+-----+-----+-----+
| 4 | George Boole | Professor of Mathematics | Math/Logic | 50 | 12573 | 51 |
+-----+-----+-----+-----+-----+-----+
| 5 | Mikhail Vasilyevich Lomonosov | Professor of Chemistry | Physic/Biolog/Literature | 67 | 11511 | 29 |
+-----+-----+-----+-----+-----+-----+

===== MENU =====
1. Создать список учёных
2. Ввести информацию об учёном в начало
3. Ввести информацию об учёном в конец
4. Удалить учёного из списка
5. Вывести список всех учёных
6. Удалить 2 и 3 запись
0. Выход из программы
-> 0
Было удалено 2 записей...
Список пуст...
Выход из программы...
```

Рисунок 1.13 – Результат работы программы на языке С

ВЫВОД

В ходе лабораторной работы были исследованы особенности представления и обработки списков структур данных в языке C/C++. На практике были успешно реализованы алгоритмы работы с линейными списками структур данных, включая их создание, обработку и освобождение памяти под список структур. Экспериментально подтверждена и изучена неразрывная связь между указателями и линейными списками структур данных. Приобретены навыки эффективного использования динамического выделения памяти под линейные списки структур данных на языках C/C++.